

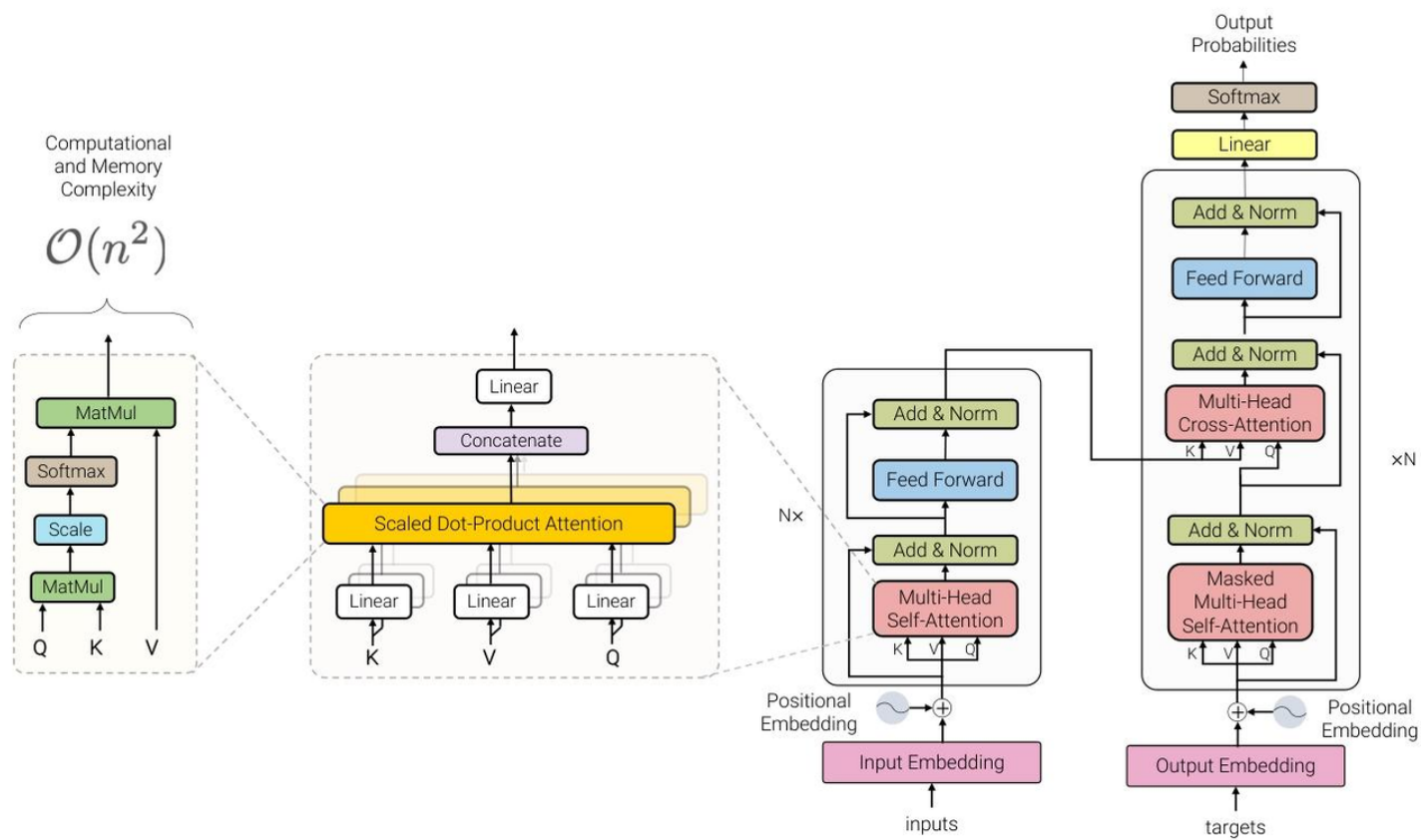
大模型架构性能优化



1. Transformer 的架构有哪些性能瓶颈？
2. 业内有哪些针对性的优化手段？
3. 有可能会有那些新的算法架构（非transformer）突破瓶颈？

1. 架构性能瓶颈

1.1 重新认识Transformer 基础架构



1. 核心组件：输入嵌入（Input Embedding）、自注意力机制（Self-Attention）、前馈神经网络（Feed-Forward Network, FFN）、层归一化（Layer Normalization）与残差连接（Residual Connection）
2. 性能优化目标：算力（算法的复杂度）、内存（缓存）、速度（并行）

1.2 影响性能的核心组件

多头自注意力（Multi-Head Self-Attention）

在 Transformer 的标准架构中，多头自注意力（Multi-Head Self-Attention）模块最容易引起性能问题，具体体现在以下方面：

- **计算复杂度高**

在多头自注意力机制中，需要计算查询向量（Query）、键向量（Key）和值向量（Value）之间的点积，计算复杂度为 $O(n^2)$ ，其中 n 是输入序列的长度。

- **内存占用大**

由于自注意力机制要计算序列中每个位置与其他所有位置的关联，在计算过程中会产生大量的中间结果，这些中间结果在反向传播计算梯度时需要被保留，进而占用大量的内存。

- **并行化困难**

虽然 Transformer 在一定程度上可以进行并行计算，但多头自注意力模块中每个位置的计算依赖于其他位置的结果，这使得在并行计算时存在一定的限制。与卷积神经网络（CNN）相比，CNN 中的卷积操作可以很容易地在不同位置上并行计算，而多头自注意力模块的并行化难度更大，难以充分利用现代硬件（如 GPU）的并行计算能力，从而影响整体计算效率。

前馈神经网络（Feed Forward）

此外，Transformer 中的前馈神经网络（Feed Forward）模块虽然计算量也不小，但它的计算复杂度是线性的，随着序列长度增加，计算量增长相对缓慢，内存占用和并行化方面的问题也不如多头自注意力模块突出。层归一化（Add & Norm）和线性变换（Linear）等模块的计算复杂度和内存需求相对较低，一般不会成为性能瓶颈。

1.3 性能分析实例

为了量化 Transformer 性能瓶颈与生成 token 长度的关系（以**10B 参数模型**为例），我们从**计算复杂度**、**内存占用**和**延迟**三个维度，分析不同序列长度下的表现。以下是基于典型 10B 模型配置（如隐藏维度 $d = 4096$ 、32 个注意力头、24 层、批量大小 $\text{batch size} = 1$ ）的结构化分析：

1.3.1 计算复杂度

- **自注意力机制：**

标准自注意力的复杂度为 $O(N^2 d)$ 。对于 $d = 4096$ 的 10B 模型：

- 当 $N = 100$ 时：每个注意力头需 $100^2 \times 4096 = 40,960,000$ 次浮点运算（FLOPs）。
- 当 $N = 1000$ 时：每个注意力头需 $1000^2 \times 4096 = 4,096,000,000$ 次 FLOPs。

若有 32 个注意力头，自注意力总运算量从 $N = 100$ 时的约 1.3×10^9 次，增长到 $N = 1000$ 时的约 1.3×10^{11} 次——**计算量提升 100 倍**。

- **前馈神经网络（FFN）：**

FFN 包含两层线性变换（ $d \rightarrow 4d \rightarrow d$ ），复杂度为 $O(Nd^2)$ 。对于 $d = 4096$ ：

- $N = 100$ 时：每层约需 $100 \times 4096^2 \approx 1.68 \times 10^8$ 次 FLOPs。
- $N = 1000$ 时：每层约需 $1000 \times 4096^2 \approx 1.68 \times 10^{10}$ 次 FLOPs。

若有 24 层，FFN 总运算量从 $N = 100$ 时的约 4.0×10^9 次，增长到 $N = 1000$ 时的约 4.0×10^{11} 次——同样**提升 100 倍**。

1.3.2 内存占用

- **KV 缓存：**

KV 缓存用于存储每个 token 的中间 K （键）和 V （值）矩阵。对于 $d = 4096$ 和 32 个注意力头：

- 每个 token 占用内存： $2 \times 32 \times 4096 = 262,144$ 字节（32 KB）。
- $N = 100$ 时：KV 缓存总占用 $100 \times 32 \text{ KB} = 3.2 \text{ MB}$ 。
- $N = 1000$ 时：KV 缓存总占用 $1000 \times 32 \text{ KB} = 32 \text{ MB}$ 。
- $N = 10,000$ 时：KV 缓存总占用 $10,000 \times 32 \text{ KB} = 320 \text{ MB}$ 。

- **中间激活值：**

各层的中间输出（如层的输出结果）对长度为 N 的序列，每层需占用 $N \times d$ 的内存。对于 24 层和 $d = 4096$ ：

- $N = 100$ 时： $24 \times 100 \times 4096 = 983,040$ 字节（约 1 MB）。
- $N = 1000$ 时： $24 \times 1000 \times 4096 = 9,830,400$ 字节（约 10 MB）。

1.3.3 推理延迟

- **实际缩放规律：**

在 NVIDIA A100 GPU（40 GB 显存、312 TFLOPs FP16 算力）上，10B 模型的推理延迟随 N **近乎线性增长**：

- $N = 100$ 时：每条序列约 50 毫秒。
- $N = 1000$ 时：每条序列约 500 毫秒。
- $N = 10,000$ 时：每条序列约 5 秒（受内存带宽和 KV 缓存吞吐量限制）。

1.3.4 训练吞吐量

- **批量大小限制：**

训练时，更长的序列会因内存限制降低最大批量大小。对于 10B 模型：

- $N = 100$ 时：批量大小约为 32（可在 40 GB 显存中容纳）。
- $N = 1000$ 时：批量大小约为 3（内存被 KV 缓存和激活值占满）。

这使得 $N = 1000$ 时的训练吞吐量比 $N = 100$ 时**下降约 10 倍**。

1.3.5 小结

对于 10B Transformer 模型：

- 计算量和**内存**随序列长度**二次方增长**（源于自注意力的 $O(N^2)$ 特性）。
- 延迟和**吞吐量**随序列长度**线性下降**，因为计算和内存访问都成为了瓶颈。

这也量化解释了为何长序列（如 $N > 1000$ ）会严重限制大型 Transformer 模型的性能。

1.4 内存分析实例

为什么大模型的运行，动不动就需要几个GB的内存

大模型运行时需要几个GB甚至更多的内存，这是由模型的架构特点、计算过程以及数据存储需求等多方面因素共同决定的。

由于模型参数、中间计算结果、激活值、KV缓存以及优化器状态等多方面的内存需求，一个10B模型在运行时很容易就需要几个GB甚至更多的内存。

下面以10B模型为例进行说明：

1.4.1 模型参数存储

10B模型意味着大约有100亿个参数，在常见的深度学习框架中，参数通常以32位浮点数（float32）的形式存储，每个float32数据占用4字节的内存。

- 假设模型参数均匀分布在各个层中，以Transformer架构为例，它包含多个层，如自注意力层和前馈神经网络层等。仅仅存储这些参数就需要占用大量内存。例如，100亿个参数就需要 $100000000000 \times 4 = 400000000000$ 字节，换算后约为 37.25GB （ $1\text{GB} = 1024 \times 1024 \times 1024$ 字节）。

1.4.2 中间计算结果存储

在模型运行过程中，会产生大量的中间计算结果，比如：

- **自注意力机制计算：**在Transformer的自注意力计算中，要生成查询（Q）、键（K）、值（V）矩阵。假设输入序列长度为 N ，隐藏层维度为 d （对于10B模型， d 可能为4096），则Q、K、V矩阵的大小均为 $N \times d$ 。当 $N = 1000$ 时，仅这三个矩阵占用的内存就达到 $3 \times 1000 \times 4096 \times 4 = 49152000$ 字节，约为 46.875MB（不考虑多头注意力的情况下，若有32个注意力头，内存占用还要再乘以32）。而且在计算注意力分数时，还会生成大小为 $N \times N$ 的分数矩阵，同样会占用内存。
- **前馈神经网络计算：**前馈神经网络部分通常有两层线性变换，从 d 维映射到 $4d$ 维，再映射回 d 维。在这个过程中，中间层的输出等结果都需要在内存中暂存。对于长度为 N 的序列，中间层 $4d$ 维的输出数据占用内存为 $N \times 4d \times 4$ 字节。当 $N = 1000$ ， $d = 4096$ 时，这部分内存占用达到 $1000 \times 4 \times 4096 \times 4 = 65536000$ 字节，约为 62.5MB。

1.4.3 激活值存储

模型各层的激活值（即经过非线性激活函数后的输出值）也需要占用内存。对于一个具有多层结构的10B模型，每一层都会产生激活值。

- 以一个包含24层的Transformer架构10B模型为例，每层激活值的大小与输入序列长度和隐藏层维度有关。假设输入序列长度 $N = 1000$ ，隐藏层维度 $d = 4096$ ，那么每层激活值占用内存为 $N \times d \times 4 = 1000 \times 4096 \times 4 = 16384000$ 字节，约为 15.625MB，24层的激活值总共占用内存约为 $15.625 \times 24 = 375$ MB。

1.4.4 KV缓存存储

在推理过程中，为了提高效率，通常会使用KV缓存技术来存储键（K）和值（V）向量。对于每个token，都要存储对应的K和V向量。

- 假设模型有32个注意力头，隐藏层维度 $d = 4096$ ，那么每个token的KV缓存占用内存为 $2 \times 32 \times 4096 \times 4 = 1048576$ 字节，即 1MB。当输入序列长度 $N = 1000$ 时，KV缓存占用内存就达到 $1000 \times 1 = 1000$ MB，也就是 1GB。

1.4.5 优化器状态存储

在模型训练过程中，优化器（如Adam优化器）需要存储一些状态信息，比如每个参数的一阶矩估计和二阶矩估计等。

- 对于10B模型庞大的参数数量，优化器状态占用的内存也不容小觑。以Adam优化器为例，每个参数需要额外存储两个状态变量，这就使得内存占用又增加了一倍左右，对于10B模型来说，这部分内存占用也能达到几个GB。

2. 参考文献

<https://github.com/weigao266/Awesome-Efficient-Arch>

1. A survey of large language models. <https://arxiv.org/abs/2303.18223>
2. Language models are few-shot learners.
<https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html>
3. Exploring the limits of transfer learning with a unified text-to-text transformer.
<http://jmlr.org/papers/v21/20-074.html>
4. Codebert: A pre-trained model for programming and natural languages.
<https://arxiv.org/abs/2002.08155>
5. Competition-level code generation with alphacode.
<https://www.science.org/doi/10.1126/science.abq4747>
6. InCoder: A generative model for code infilling and synthesis. <https://arxiv.org/abs/2204.05999>
7. Retrieval augmented language model pre-training.
<http://proceedings.mlr.press/v119/guu20a.html>
8. Neural machine translation for low-resource languages: A survey.
<https://dl.acm.org/doi/abs/10.1145/3579068>
9. Improving language understanding by generative pre-training.
<https://paperswithcode.com/method/gpt>
10. Language models are unsupervised multitask learners.
<https://openai.com/research/language-models-are-unsupervised-multitask-learners>
11. GPT-4 technical report. <https://arxiv.org/abs/2303.08774>
12. GPT-4O system card. <https://arxiv.org/abs/2410.21276>
13. OpenAI O1 system card. <https://arxiv.org/abs/2412.16720>
14. OpenAI O3 and O4-mini system card. Technical report, <https://www.openai.com/research>, April 2025.
15. GPT-OSS-120B & GPT-OSS-20B model card. Technical report, <https://www.openai.com/research>, August 2025.
16. Introducing GPT-5. Blog post, <https://www.openai.com/blog>, August 7, 2025.
17. Introducing Claude. Blog post, <https://anthropic.com/blog>, March 14, 2023.
18. Introducing Claude 2.1. Blog post, <https://anthropic.com/blog>, November 21, 2023.
19. Claude 3.7 Sonnet and Claude Code. Blog post, <https://anthropic.com/blog>, February 25, 2025.
20. Introducing Claude 4. Blog post, <https://anthropic.com/blog>, May 23, 2025.
21. Claude Opus 4.1. Blog post, <https://anthropic.com/blog>, August 6, 2025.

22. Gemini: A family of highly capable multimodal models. <https://arxiv.org/abs/2312.11805>
23. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. <https://arxiv.org/abs/2403.05530>
24. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. <https://deepmind.google/teams/gemini/>
25. DeepSeek LLM: Scaling open-source language models with longtermism. <https://arxiv.org/abs/2401.02954>
26. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. <https://arxiv.org/abs/2405.04434>
27. DeepSeek-V3 technical report. <https://arxiv.org/abs/2412.19437>
28. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. <https://arxiv.org/abs/2501.12948>
29. Qwen technical report. <https://arxiv.org/abs/2309.16609>
30. Qwen2 technical report. <https://arxiv.org/abs/2412.15115>